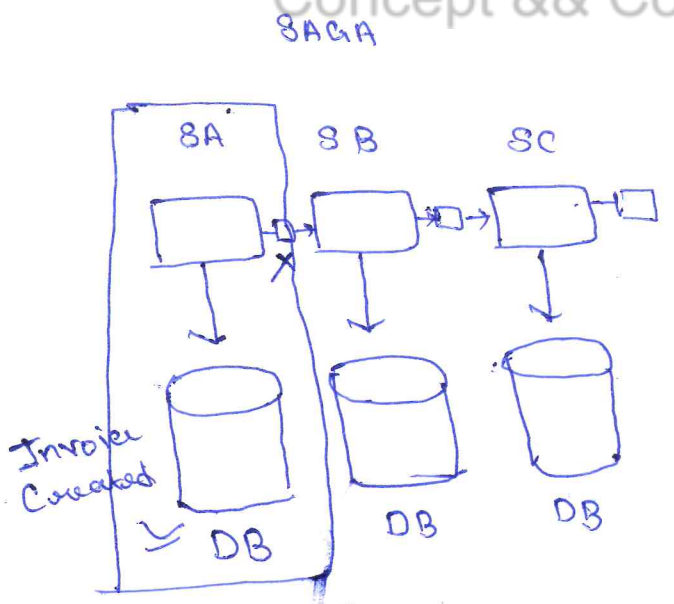
The Dual Write Problem

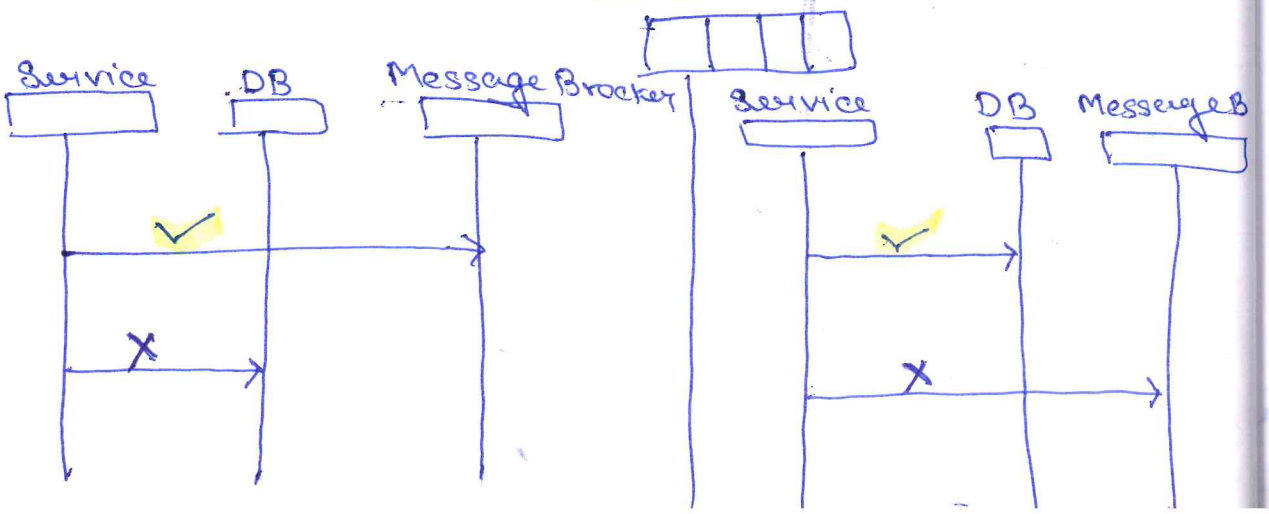
The Dual Write Problem occurs when a component needs to persist a change in two different systems say DB and Message broker like kafk.

We need to make sure that both are the consistent state. There should not be any case when DB is sucess but event publishing gat faild or vice-versa.

SAGA
↓
Dual writ problem

Concept && Coding





How to resolve this?

The first things comes to mind in : 2PC (Two-Phase Commit)

- In this 2PC, there is a coordinator,
- Which sends Prepare message to all the participant services. And wait for Ack.
- Once Ack received then it will send Commit message and again ~~waits~~ waits for the Ack message

This makes sure that, all the participant services has properly committed the txn.

But.....

2PC is not a viable Solution for Dual Write Problem

Why so??

Concept && Coding

- Dual Write scenarios may involves heterogenous system like (DB + Message Queue). And some Message Queue or Broker do not support 2PC.
- In Distributed system, 2PC is slow (gives high latency), as it provides Strong Consistency by waiting for all the participants to Commit.
- If 2PC coordinator crashes participants may stuck in an uncertain state. Till it recovered.

So, how to resolve the DUAL WRITE PROBLEM

Answer is : Event Driven Architecture Patterns

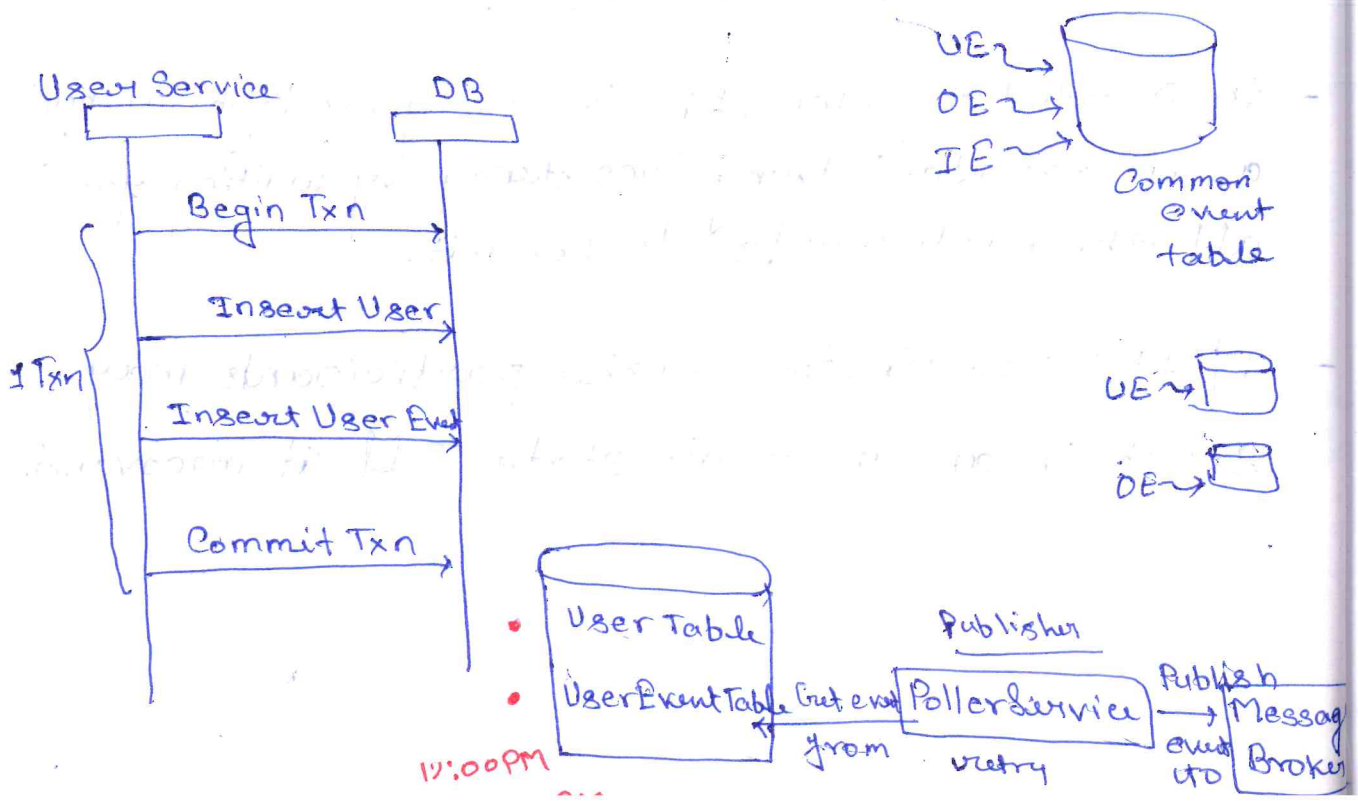
1. Transactional Outbox Pattern
2. Listen to yourself Pattern
3. Transactional Log Tailing Pattern

1. Transactional Outbox Pattern :-

- In this patten, instead of writing in both DB and publishing an event into a seprate table called OUTBOX table.

Concept && Coding

- Later, a separate process (pollar) reads the Outbox table, and publish the event to the message broker in asyne manner.



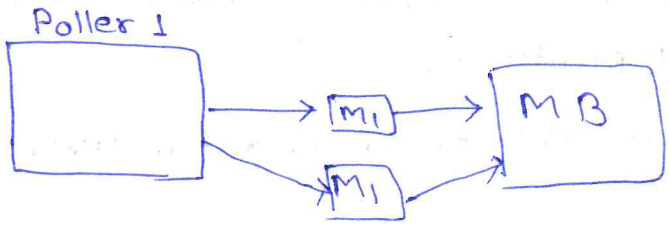
④ Outbox table is regularly cleaned up as when messages are successfully published.

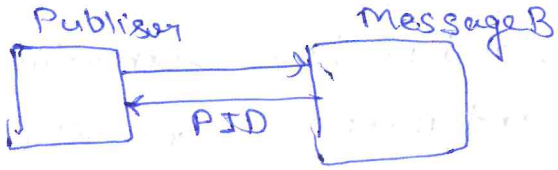
Challenges

- Extra effort to write this Poller service.
- Delay between writing an event and publishing it to Broker Might take time.
- A Poller may ~~push~~ publish the event multiple times to Broker. So Idempotency should be handled.

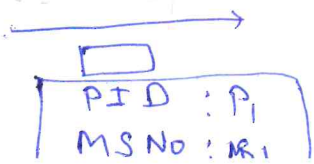
1. Producer need to set "enabled.idempotency = true"
2. Then Kafka, assigns a producer id to each producer when they starts.
3. Also every message sent from the producer now has a monotonically increasing sequencing number.
4. If kafka ~~seen~~ sees dublicate (producer id + sequence no) it discard the event automatically

Concept && Coding

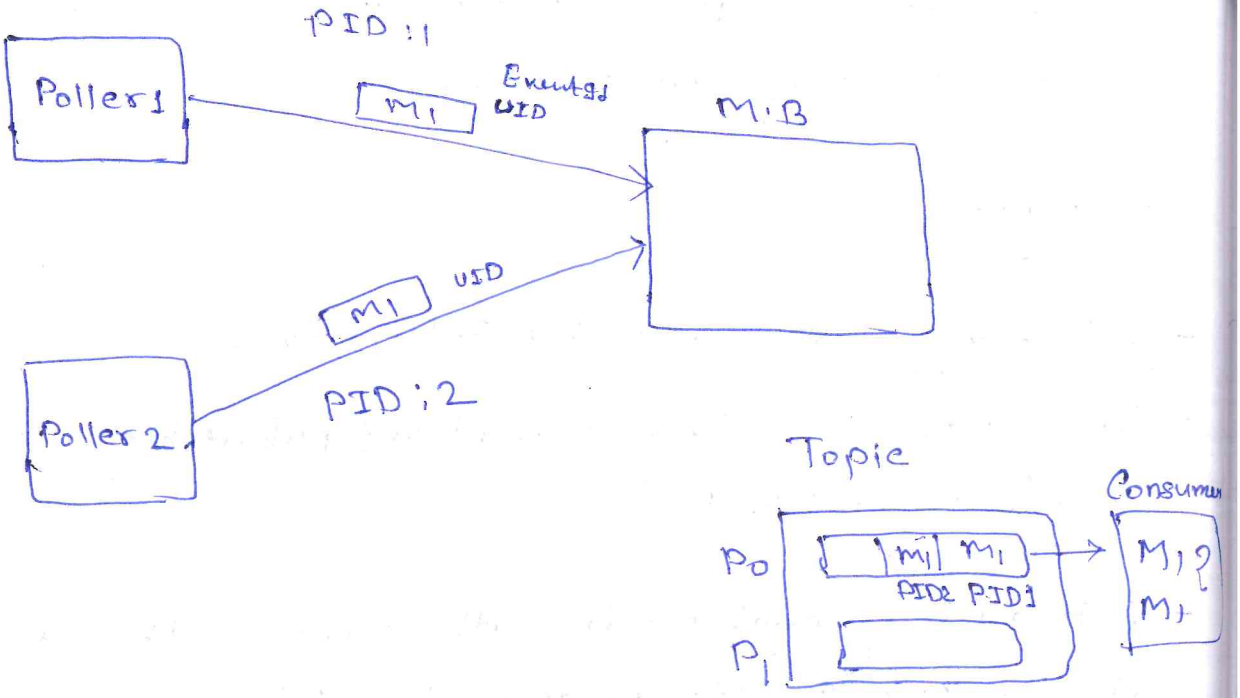




enabled.idmpotency = tr



(5)



- As Multiple Poller can pick same events and cause duplication of events published.

1. Now this will not help "enable.idempotency = true". Because different producer will get different PID.

Concept && Coding

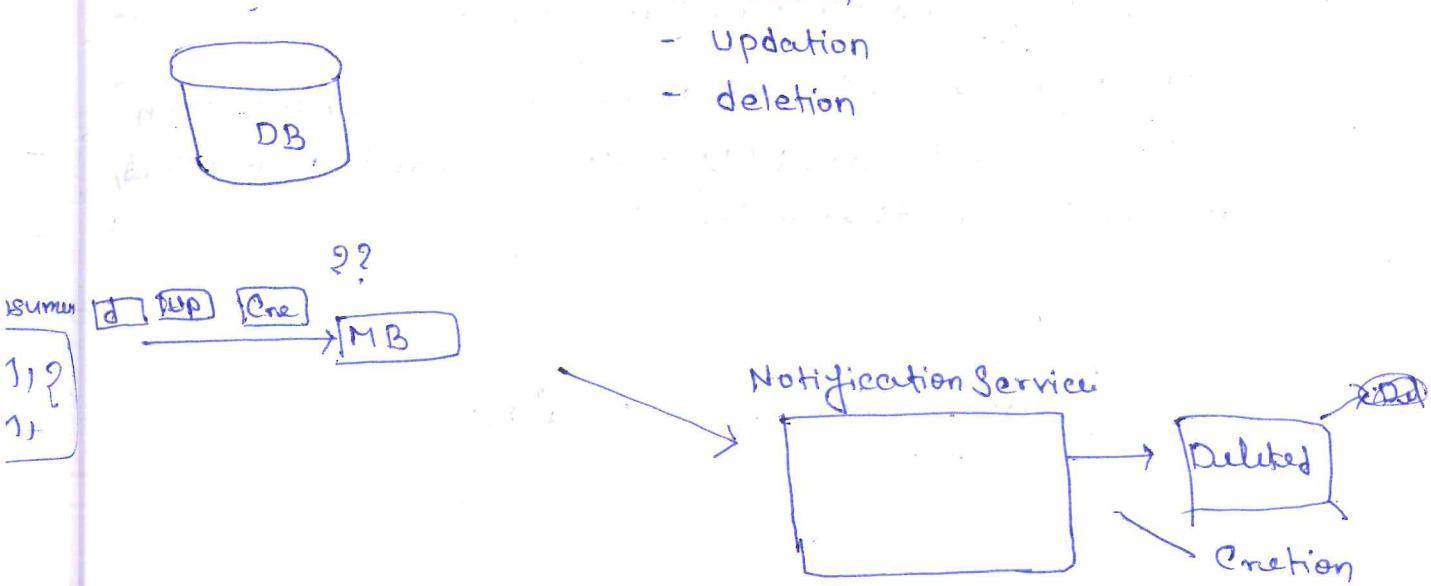
2. So, at Consumer side Idempotency should be handled too, in cache they can store the unique ids of events which they have processed.

☆ - Events might be publised Out of Order if different pollers picking event parallel.

1. Kafka only guarantees order if there is only 1 producer and 1 partition.
2. If there are multiple producers and multiple partitions, then ordering is not guaranteed.
3. If Ordering is must, and we have scenarios where multiple producers and partitions are there, then Kafka Streams can be used.

Ordering Issue

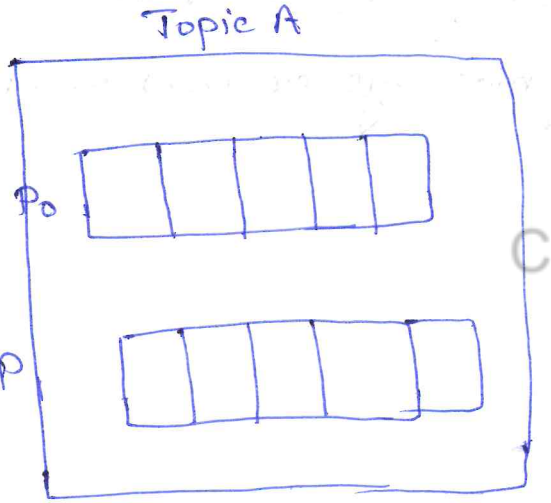
- Creation
- Updation
- deletion



Message :-

- Key → hash % no of Par
- Partion = P0
- key ×

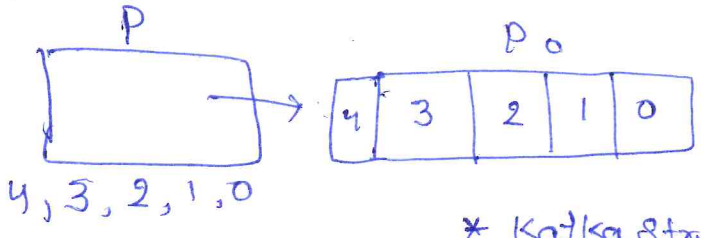
  P ×



Concept && Coding

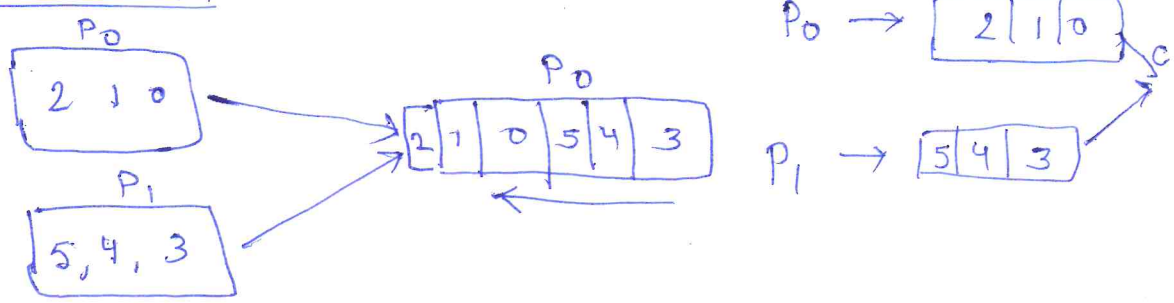
Note

1 publiser → 1 partician order is maintained.



* Kafka Stream is reodering

More Partican



- What if Poller not able to publish the event to message broker. so event should not be lost.

1. Exponential Backoff and jitter - Retry machanism
2. After retries, move to Failed event table or depend upon how outbox table is used (If status column is maintained, then it can be in that table itself).

- Outbox table can grow, should be cleaned up regularly.

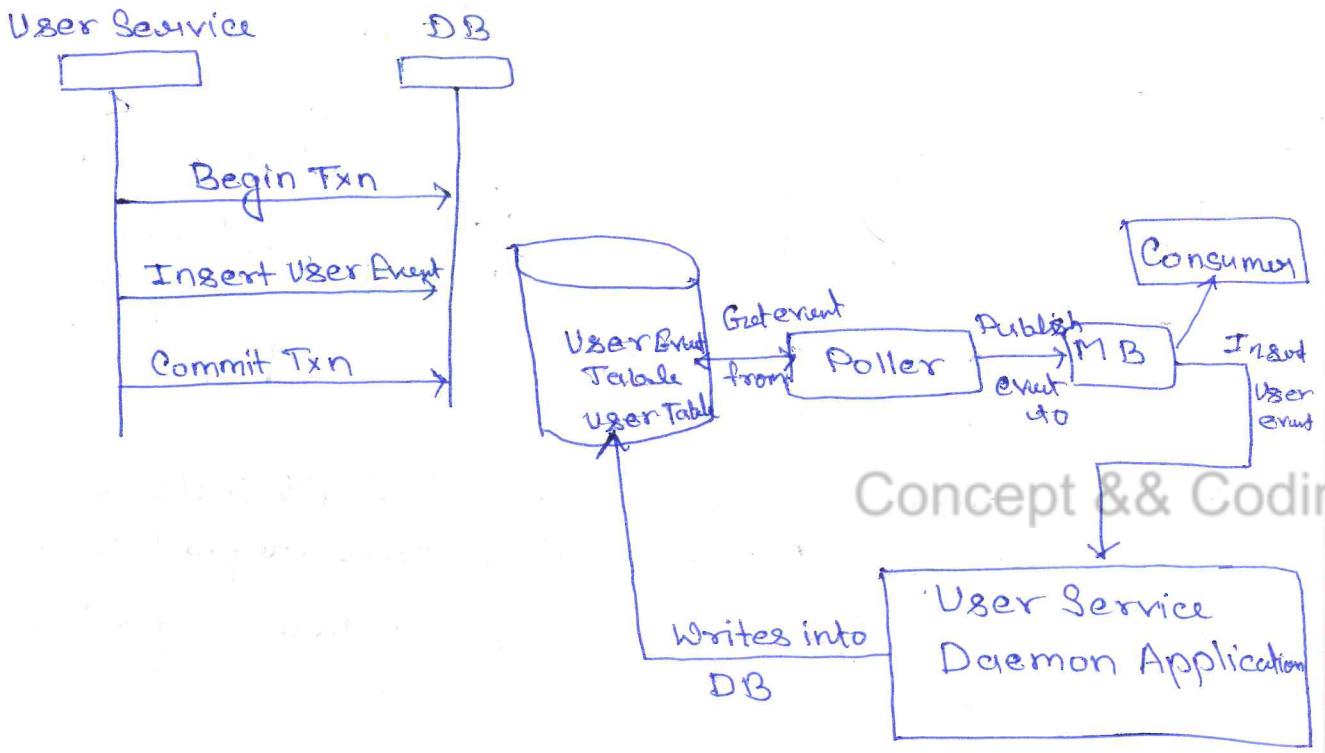
1 After every message or event published sucessfully.

2 Or batch job.

Concept && Coding

Listen to Yourself Pattern

- Just a variation of Transaction Outbox Pattern.
- Application writes only event in the Outbox table.
- Then Poller fetch the event from the Outbox table and publish an event.
- Application also listen to the same event and now update its DB.



Concept && Coding